```
> ##########################################################################
  #######
  #
  #   TauTimp -- Time-Drift Constraints (GEO, GPS, Mars)
  #   epsilon-calibrated 1D temporal probability flux
  #
  #   Author : Viorel-Mihai Popescu
  #   Version: May 2026
  #   Units  : SI
  #
  #   Structure:
  #     §1  Constants and physical parameters
  #     §2  Polynomial ansatz and PDE verification
  #     §3  Calibration of A and epsilon (Earth quantum + GPS drift)
  #     §4  Satellite drift function rate(r_sat)
  #     §5  GPS and GEO drift table
  #     §6  Mars predictions (same field + alternative amplitudes)
  #     §7  Plots:
  #           plot_drift   -- drift vs orbital radius (with sign reversal)
  #           plot_F_sun   -- F(r) along Sun-Earth-Mars line
  #           plot_F_time  -- F at Earth as function of t
  #           plot_F_3D    -- F(r,t) surface for Earth-Mars region
  #           plot_panel   -- 2x2 composite
  #     §8  Summary printf
  #
  ##########################################################################
  #######

  restart;
  interface(displayprecision = 8):
  with(plots):
  Digits := 25:

  #-------------------------------------------------------------------------
  ------
  # §1. CONSTANTS
  #-------------------------------------------------------------------------
  ------
  printf("=============================================================
  \n");
  printf("  §1. CONSTANTS AND PARAMETERS (SI)\n");
  printf("=============================================================
  \n");

  c_light := 2.99792458e8:
  G_grav  := 6.67430e-11:
  M_sun   := 1.98892e30:
  R_S_sun := 2*G_grav*M_sun/c_light^2:
```

```
r0       := 11222.35:            # 3.8 R_S_sun
R_cosmo  := 3.9228e22:           # TauTimp cosmological radius
r_earth  := 1.49e11:
r_mars   := 2.09e11:
t0_prop  := 500.0:
t1_prop  := 700.0:
tau_q    := 1.765e-19:

r_GEO    := 42171e3:
r_GPS    := 26371e3:

printf("  r0  (= 3.8 R_S_sun) = %.4f m   (R_S_sun = %.4f m)\n", r0,
R_S_sun);
printf("  R_cosmo             = %.4e m\n", R_cosmo);
printf("  r_earth             = %.4e m   (Sun-Earth)\n", r_earth);
printf("  r_mars              = %.4e m   (Sun-Mars)\n", r_mars);
printf("  t0 (Sun->Earth)     = %.1f s\n", t0_prop);
printf("  t1 (Sun->Mars)      = %.1f s\n", t1_prop);
printf("  tau_q (Earth quantum) = %.4e s\n", tau_q);
printf("  r_GEO, r_GPS = %.0f km, %.0f km\n", r_GEO/1000, r_GPS/1000);

#-----------------------------------------------------------------------
------
# §2. POLYNOMIAL ANSATZ AND PDE VERIFICATION
#-----------------------------------------------------------------------
------
printf("\n=======================================================================
\n");
printf("  §2. POLYNOMIAL ANSATZ AND PDE VERIFICATION\n");
printf("=======================================================
\n");

P_symbolic := A*((r-r0_s)*(r-R_s) + eps*t + c_s^2*t^2);
printf("\n  P(r,t) = A*[(r-r0)(r-R) + eps*t + c^2 t^2]\n");

PDE_check := diff(P_symbolic, t$2) - c_s^2*diff(P_symbolic, r$2);
PDE_check := simplify(PDE_check):
printf("  PDE check: d^2P/dt^2 - c^2 d^2P/dr^2 = %a\n", PDE_check);

BC1 := eval(P_symbolic, [r = r0_s, t = 0]):
BC2 := eval(P_symbolic, [r = R_s,  t = 0]):
printf("  BC: P(r0, 0) = %a   (should be 0)\n", simplify(BC1));
printf("  BC: P(R,  0) = %a   (should be 0)\n", simplify(BC2));

#-----------------------------------------------------------------------
------
# §3. CALIBRATION
#-----------------------------------------------------------------------
------
```

```
printf("\n=====================================================================
\n");
printf("  §3. CALIBRATION (Earth quantum + GPS drift = 0.3 microsec/day)
\n");
printf("=====================================================================
\n");

# A(eps) from Earth quantum condition F(r_earth, t0) = tau_q
A_of_eps := eps_val -> evalf(
  4*Pi*r_earth^2*tau_q /
  ((r_earth-r0)*(r_earth-R_cosmo) + eps_val*t0_prop + c_light^2*t0_prop^2)
):

# F field
F_field := (rr, tt, AA, ee) -> AA*((rr-r0)*(rr-R_cosmo) + ee*tt +
c_light^2*tt^2) /
                                (4*Pi*rr^2):

# Rate at satellite
rate_sat := proc(rsat, eps_val, A_val)
  local Fm, Fp, Fc;
  Fm := evalf(F_field(r_earth - rsat, t0_prop, A_val, eps_val));
  Fp := evalf(F_field(r_earth + rsat, t0_prop, A_val, eps_val));
  Fc := evalf(F_field(r_earth,        t0_prop, A_val, eps_val));
  return (Fp - Fm)/Fc;
end proc:

# Calibration: rate_sat(r_GPS) = 0.3e-6/86400
target_rate := evalf(0.3e-6/86400):
obj := eps_val -> rate_sat(r_GPS, eps_val, A_of_eps(eps_val)) -
target_rate:

eps_cal := fsolve(obj(epsv) = 0, epsv = 5.844e30 .. 5.852e30):
A_cal   := A_of_eps(eps_cal):

printf("\n  CALIBRATED VALUES (need >= 11 significant digits!):\n");
printf("    epsilon = %.12e m^2/s\n", eps_cal);
printf("    A       = %.10e s/m^2\n", A_cal);

# Verifications
printf("\n  Verifications:\n");
printf("    F(r_earth, t0)  = %.6e s   (target tau_q = %.4e)\n",
       evalf(F_field(r_earth, t0_prop, A_cal, eps_cal)), tau_q);
printf("    GPS drift / day = %.6f microsec/day (target 0.300)\n",
       rate_sat(r_GPS, eps_cal, A_cal)*86400*1e6);

# Sensitivity analysis
delta_eps := 1e22:
sens := (rate_sat(r_GPS, eps_cal+delta_eps, A_of_eps(eps_cal+delta_eps)) -
```

```
         rate_sat(r_GPS, eps_cal-delta_eps, A_of_eps(eps_cal-delta_eps)))
        /(2*delta_eps)*86400*1e6:
printf("\n  Calibration sensitivity:\n");
printf("    d(GPS drift)/d(epsilon) = %.3e microsec/day per (m^2/s)\n",
sens);
printf("    A 1e-6 shift in last digit shifts drift by %.3e
microsec/day\n",
       sens*1e25);

#-----------------------------------------------------------------------
------
# §4. SATELLITE DRIFT FUNCTION
#-----------------------------------------------------------------------
------
printf("\n=======================================================
\n");
printf("  §4. SATELLITE DRIFT FUNCTION\n");
printf("=========================================================
\n");

# Convenience: drift per day for a given r_sat
drift_day_uS := rsat -> rate_sat(rsat, eps_cal, A_cal)*86400*1e6:
drift_h_uS   := rsat -> rate_sat(rsat, eps_cal, A_cal)*3600*1e6:

printf("\n  drift_day_uS(r_sat) returns drift in microsec/day\n");
printf("  drift_h_uS(r_sat)   returns drift in microsec/h\n");

#-----------------------------------------------------------------------
------
# §5. GPS AND GEO DRIFT TABLE
#-----------------------------------------------------------------------
------
printf("\n=======================================================
\n");
printf("  §5. SATELLITE DRIFT TABLE (calibrated)\n");
printf("=========================================================
\n");

printf("\n  %-10s %-12s %-16s %-16s\n", "Satellite", "r_sat [km]", "drift
[µs/h]", "drift [µs/day]");
printf("  %s\n", "-----------------------------------------------------
----");

for sat_data in [["GPS", r_GPS], ["GEO", r_GEO]] do
  printf("  %-10s %-12.0f %-16.6f %-16.6f\n",
         sat_data[1], sat_data[2]/1000.0,
         drift_h_uS(sat_data[2]),
         drift_day_uS(sat_data[2]));
end do:
```

```
printf("\n  Extended sweep over orbital radius:\n");
printf("  %-12s %-16s %-16s\n", "r_sat [km]", "drift [µs/h]", "drift
[µs/day]");
printf("  %s\n", "-----------------------------------------------------");
for rs_km in [1000, 5000, 10000, 20000, 22000, 26371, 30000, 42171, 50000]
do
  rs := rs_km*1e3:
  printf("  %-12.0f %-+16.6f %-+16.6f\n",
         rs_km, drift_h_uS(rs), drift_day_uS(rs));
end do:

# Find sign reversal (zero crossing) numerically
printf("\n  Finding drift sign reversal:\n");
zero_rs := fsolve(drift_day_uS(x) = 0, x = 15e6 .. 25e6):
printf("    Zero crossing at r_sat = %.2f km\n", zero_rs/1000.0);

#-----------------------------------------------------------------------
------
# §6. MARS PREDICTIONS
#-----------------------------------------------------------------------
------
printf("\n=====================================================================
\n");
printf("  §6. MARS PREDICTIONS\n");
printf("=====================================================================
\n");

# Same-field prediction
F_mars_same := evalf(F_field(r_mars, t1_prop, A_cal, eps_cal)):
printf("\n  Same-field prediction:\n");
printf("    F(r_mars, t1) = %.6e s\n", F_mars_same);
printf("    ratio F_mars / tau_q = %.6f\n", F_mars_same/tau_q);
printf("    Mars quantum is %s than Earth quantum\n",
       `if`(F_mars_same > tau_q, "LARGER", "SMALLER"));

# Cumulative offset
cumul_offset := (1 - F_mars_same/tau_q)*86400:
printf("    Cumulative Mars-Earth offset = %.2f s/day (%.2f h/day)\n",
       cumul_offset, cumul_offset/3600);

# Alternative: independent A for larger Mars quantum
tau_M1 := tau_q*r_mars/r_earth:
tau_M2 := tau_q*(r_mars/r_earth)^2:
A_M1 := evalf(4*Pi*r_mars^2*tau_M1 /
              ((r_mars-r0)*(r_mars-R_cosmo) + eps_cal*t1_prop + c_light^2*
t1_prop^2)):
A_M2 := evalf(4*Pi*r_mars^2*tau_M2 /
              ((r_mars-r0)*(r_mars-R_cosmo) + eps_cal*t1_prop + c_light^2*
t1_prop^2)):
```

```
printf("\n  Alternative Mars amplitudes (slower-flux conventions):\n");
printf("    Conv v1 [tau_M = tau_q (r_M/r_E)]:    tau_M = %.4e s, A_M =
%.4e\n",
       tau_M1, A_M1);
printf("    Conv v2 [tau_M = tau_q (r_M/r_E)^2]:  tau_M = %.4e s, A_M =
%.4e\n",
       tau_M2, A_M2);

# Mars-orbit satellite drifts (same-field calibration)
rate_sat_mars := proc(rsat_M, eps_val, A_val)
  local Fm, Fp, Fc;
  Fm := evalf(F_field(r_mars - rsat_M, t1_prop, A_val, eps_val));
  Fp := evalf(F_field(r_mars + rsat_M, t1_prop, A_val, eps_val));
  Fc := evalf(F_field(r_mars,          t1_prop, A_val, eps_val));
  return (Fp - Fm)/Fc;
end proc:

printf("\n  Mars-orbiting satellite drifts (same-field):\n");
printf("  %-22s %-12s %-16s\n", "Orbit type", "r_sat [km]", "drift
[μs/day]");
printf("  %s\n", "----------------------------------------------------")
;
for mo_data in [
  ["Low Mars orbit",     4000],
  ["Areostationary",    17032],
  ["High Mars orbit",   30000]
] do
  rs := mo_data[2]*1e3:
  d := rate_sat_mars(rs, eps_cal, A_cal)*86400*1e6:
  printf("  %-22s %-12.0f %+16.6f\n", mo_data[1], mo_data[2], d);
end do:

#------------------------------------------------------------------------
------
# §7. STRUCTURED PLOTS
#------------------------------------------------------------------------
------
printf("\n========================================================================
\n");
printf("  §7. STRUCTURED PLOTS\n");
printf("========================================================================
\n");

# ----- Plot 1: drift vs orbital radius (full sweep) -----
# Highlight GPS (calibration) and GEO predictions

# Curve
plot_drift_curve := plot(
  drift_day_uS(rho), rho = 1e6 .. 5e7,
```

```
  color = "DarkBlue", thickness = 2);

# Highlight points
gps_pt := plots:-pointplot([[r_GPS, drift_day_uS(r_GPS)]],
                           symbol = solidcircle, symbolsize = 22,
                           color = "Crimson"):
geo_pt := plots:-pointplot([[r_GEO, drift_day_uS(r_GEO)]],
                           symbol = solidcircle, symbolsize = 22,
                           color = "ForestGreen"):

# Labels for the points
gps_label := plots:-textplot([r_GPS, drift_day_uS(r_GPS) + 0.8, "GPS
(0.300 µs/day)"],
                             font = ["HELVETICA", 11], align = right):
geo_label := plots:-textplot([r_GEO, drift_day_uS(r_GEO) + 0.8, "GEO (5.25
µs/day)"],
                             font = ["HELVETICA", 11], align = left):

# Zero line
zero_line := plot(0, rho = 1e6 .. 5e7, color = "Gray", linestyle = 3):

plot_drift := plots:-display(
  [plot_drift_curve, zero_line, gps_pt, geo_pt, gps_label, geo_label],
  title = "Time-drift vs orbital radius (calibrated TauTimp)",
  labels = ["orbital radius r_sat [m]", "drift [µs/day]"],
  axes = boxed
):
printf("\n  plot_drift   :  drift [µs/day] vs r_sat, with GPS+GEO
markers\n");
printf("                  display(plot_drift);\n");

# ----- Plot 2: F(r) along Sun -> Earth -> Mars line at t = t0 -----
F_radial_t0 := r_arg -> evalf(F_field(r_arg, t0_prop, A_cal, eps_cal)):
plot_F_curve := plot(F_radial_t0(r), r = 0.5*r_earth .. 1.5*r_mars,
                     color = red, thickness = 2):
earth_marker := plots:-pointplot([[r_earth, F_radial_t0(r_earth)]],
                                 symbol = diamond, symbolsize = 18,
                                 color = "DarkOrange"):
mars_marker := plots:-pointplot([[r_mars, F_radial_t0(r_mars)]],
                                symbol = diamond, symbolsize = 18,
                                color = "DarkRed"):
plot_F_sun := plots:-display(
  [plot_F_curve, earth_marker, mars_marker],
  title = "Radial profile F(r) at t = t0 = 500s",
  labels = ["r [m]", "F(r,t0) [s]"],
  axes = boxed
):
printf("  plot_F_sun   :  F(r) along Sun-Earth-Mars line at t=t0\n");
printf("                  display(plot_F_sun);\n");
```

```
# ----- Plot 3: F(r_earth, t) -- temporal evolution at Earth -----
F_temporal_earth := t_arg -> evalf(F_field(r_earth, t_arg, A_cal, eps_cal)
):
plot_F_time := plot(F_temporal_earth(t), t = 0 .. 1500,
                    color = blue, thickness = 2,
                    title = "F(r_earth, t) -- temporal evolution at
Earth",
                    labels = ["t [s]", "F [s]"]):
printf("  plot_F_time  :  F(r_earth, t) vs t\n");
printf("                  display(plot_F_time);\n");

# ----- Plot 4: 3D F(r,t) surface for Earth-Mars region -----
plot_F_3D := plot3d(evalf(F_field(r, t, A_cal, eps_cal)),
                    r = r_earth .. r_mars,
                    t = 0 .. 1000,
                    title = "F(r,t) surface -- Earth-Mars region",
                    axes = boxed,
                    shading = zhue):
printf("  plot_F_3D    :  F(r,t) surface\n");
printf("                  display(plot_F_3D);\n");

# ----- Plot 5: hourly drift vs r_sat (zoomed near GPS/GEO) -----
plot_drift_h := plot(drift_h_uS(rho), rho = 5e6 .. 5e7,
                     color = "Purple", thickness = 2,
                     title = "Hourly drift vs r_sat (zoom GPS/GEO)",
                     labels = ["r_sat [m]", "drift [μs/h]"]):
printf("  plot_drift_h :  hourly drift vs r_sat\n");
printf("                  display(plot_drift_h);\n");

# ----- Plot 6: Mars-orbit drift -----
rate_mars_orbit := rsM -> rate_sat_mars(rsM, eps_cal, A_cal)*86400*1e6:
plot_mars_drift := plot(rate_mars_orbit(rho), rho = 1e6 .. 4e7,
                        color = "DarkRed", thickness = 2,
                        title = "Time-drift around Mars vs r_sat^M",
                        labels = ["r_sat^M [m]", "drift [μs/day]"]):
printf("  plot_mars_drift : drift around Mars vs r_sat (same-field)\n");
printf("                  display(plot_mars_drift);\n");

# ----- Composite panel 2x3 -----
plot_panel := plots:-display(
  Matrix(2, 3, [
    [plot_drift,    plot_F_sun,     plot_F_time],
    [plot_drift_h,  plot_mars_drift, plot_F_3D]
  ])
):
printf("\n  plot_panel   :  composite 2x3 panel\n");
printf("                  display(plot_panel);\n");

#-----------------------------------------------------------------------
```

```
------
# §8. SUMMARY
#-------------------------------------------------------------------
------
printf("\n=================================================================
\n");
printf("  §8. SUMMARY OF TIME-DRIFT CONSTRAINTS\n");
printf("=================================================================
\n");

printf("\n  Calibrated parameters:\n");
printf("    epsilon = %.12e m^2/s\n", eps_cal);
printf("    A       = %.10e s/m^2\n", A_cal);

printf("\n  Earth-orbit satellites (Sun-Earth line):\n");
printf("    GPS  (r=%.0f km): drift = %+.6f μs/h = %+.6f μs/day
[calibration]\n",
       r_GPS/1000, drift_h_uS(r_GPS), drift_day_uS(r_GPS));
printf("    GEO  (r=%.0f km): drift = %+.6f μs/h = %+.6f μs/day
[prediction]\n",
       r_GEO/1000, drift_h_uS(r_GEO), drift_day_uS(r_GEO));
printf("    Sign reversal at r_sat = %.2f km\n", zero_rs/1000);

printf("\n  Mars predictions (same field):\n");
printf("    F(Mars,t1) = %.4e s (%.4f tau_q)\n", F_mars_same,
F_mars_same/tau_q);
printf("    Cumulative Mars-Earth offset = %.2f h/day\n",
cumul_offset/3600);

printf("\n  Mars alternative amplitudes:\n");
printf("    Conv v1 (linear):    A_M = %.3e\n", A_M1);
printf("    Conv v2 (quadratic): A_M = %.3e\n", A_M2);

printf("\n=================================================================
\n");
printf("  END OF TRANSCRIPT\n");
printf("=================================================================
\n");

# End of file
```

```
================================================================
  §1. CONSTANTS AND PARAMETERS (SI)
================================================================
  r0  (= 3.8 R_S_sun) = 11222.3500 m   (R_S_sun = 2954.0077 m)
  R_cosmo             = 3.9228e+22 m
  r_earth             = 1.4900e+11 m   (Sun-Earth)
  r_mars              = 2.0900e+11 m   (Sun-Mars)
  t0 (Sun->Earth)     = 500.0 s
```

```
  t1 (Sun->Mars)       = 700.0 s
  tau_q (Earth quantum) = 1.7650e-19 s
  r_GEO, r_GPS = 42171 km, 26371 km

================================================================
  §2. POLYNOMIAL ANSATZ AND PDE VERIFICATION
================================================================
```

$$P_symbolic := A\left((r - r0_s)\,(r - R_s) + eps\,t + c_s^2\,t^2\right)$$

```
  P(r,t) = A*[(r-r0)(r-R) + eps*t + c^2 t^2]
```

$$PDE_check := 0$$

```
  PDE check: d^2P/dt^2 - c^2 d^2P/dr^2 = 0
  BC: P(r0, 0) = 0   (should be 0)
  BC: P(R,  0) = 0   (should be 0)

================================================================
  §3. CALIBRATION (Earth quantum + GPS drift = 0.3 microsec/day)
================================================================

  CALIBRATED VALUES (need >= 11 significant digits!):
    epsilon = 5.844970965073e+30 m^2/s
    A       = -1.6849020540e-29 s/m^2

  Verifications:
    F(r_earth, t0)  = 1.765000e-19 s   (target tau_q = 1.7650e-19)
    GPS drift / day = 0.300000 microsec/day (target 0.300)

  Calibration sensitivity:
    d(GPS drift)/d(epsilon) = 1.046e-23 microsec/day per (m^2/s)
    A 1e-6 shift in last digit shifts drift by 1.046e+02 microsec/day

================================================================
  §4. SATELLITE DRIFT FUNCTION
================================================================

  drift_day_uS(r_sat) returns drift in microsec/day
  drift_h_uS(r_sat)   returns drift in microsec/h

================================================================
  §5. SATELLITE DRIFT TABLE (calibrated)
================================================================

  Satellite  r_sat [km]   drift [μs/h]    drift [μs/day]
  ------------------------------------------------------
  GPS        26371        0.012500         0.300000
  GEO        42171        0.218796         5.251109
```

```
  Extended sweep over orbital radius:
  r_sat [km]   drift [μs/h]    drift [μs/day]
  ------------------------------------------------
  1000         -0.002549        -0.061175
  5000         -0.012222        -0.293337
  10000        -0.021180        -0.508317
  20000        -0.016241        -0.389781
  22000        -0.009820        -0.235688
  26371        +0.012500        +0.300000
  30000        +0.040936        +0.982462
  42171        +0.218796        +5.251109
  50000        +0.416479        +9.995488

  Finding drift sign reversal:
    Zero crossing at r_sat = 24218.62 km

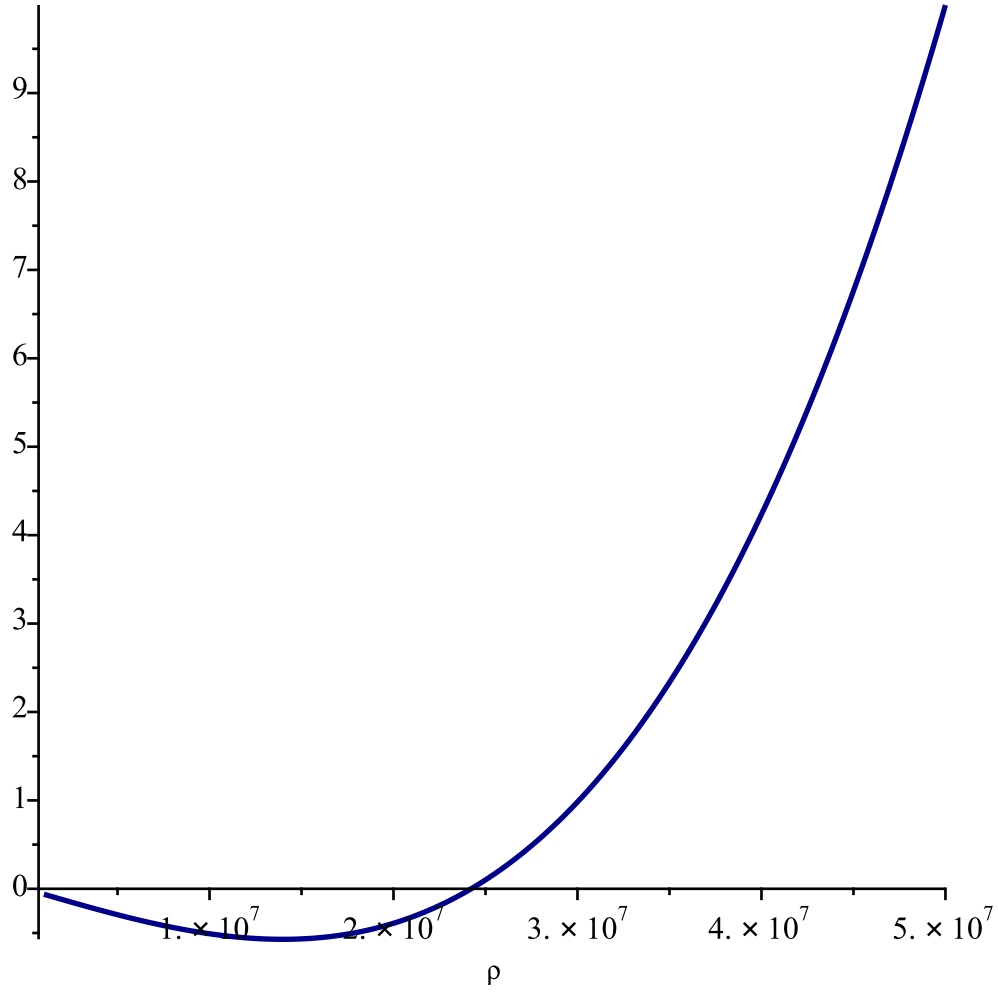
================================================================
  §6. MARS PREDICTIONS
================================================================

  Same-field prediction:
    F(r_mars, t1) = 1.260710e-19 s
    ratio F_mars / tau_q = 0.714283
    Mars quantum is SMALLER than Earth quantum
    Cumulative Mars-Earth offset = 24685.94 s/day (6.86 h/day)

  Alternative Mars amplitudes (slower-flux conventions):
    Conv v1 [tau_M = tau_q (r_M/r_E)]:    tau_M = 2.4757e-19 s, A_M =
-3.3088e-29
    Conv v2 [tau_M = tau_q (r_M/r_E)^2]:  tau_M = 3.4727e-19 s, A_M =
-4.6411e-29

  Mars-orbiting satellite drifts (same-field):
  Orbit type             r_sat [km]   drift [μs/day]
  ------------------------------------------------------
  Low Mars orbit         4000            -12635.323642
  Areostationary         17032           -53801.032356
  High Mars orbit        30000           -94763.929123

================================================================
  §7. STRUCTURED PLOTS
================================================================
```




```
  plot_drift    :  drift [μs/day] vs r_sat, with GPS+GEO markers
                   display(plot_drift);
  plot_F_sun    :  F(r) along Sun-Earth-Mars line at t=t0
                   display(plot_F_sun);
  plot_F_time   :  F(r_earth, t) vs t
                   display(plot_F_time);
  plot_F_3D     :  F(r,t) surface
                   display(plot_F_3D);
  plot_drift_h  :  hourly drift vs r_sat
                   display(plot_drift_h);
  plot_mars_drift : drift around Mars vs r_sat (same-field)
                   display(plot_mars_drift);

  plot_panel    :  composite 2x3 panel
                   display(plot_panel);

================================================================
  §8. SUMMARY OF TIME-DRIFT CONSTRAINTS
================================================================

  Calibrated parameters:
    epsilon = 5.844970965073e+30 m^2/s
    A       = -1.6849020540e-29 s/m^2
```

```
  Earth-orbit satellites (Sun-Earth line):
    GPS  (r=26371 km): drift = +0.012500 μs/h = +0.300000 μs/day
[calibration]
    GEO  (r=42171 km): drift = +0.218796 μs/h = +5.251109 μs/day  [prediction]
    Sign reversal at r_sat = 24218.62 km

  Mars predictions (same field):
    F(Mars,t1) = 1.2607e-19 s (0.7143 tau_q)
    Cumulative Mars-Earth offset = 6.86 h/day

  Mars alternative amplitudes:
    Conv v1 (linear):    A_M = -3.309e-29
    Conv v2 (quadratic): A_M = -4.641e-29

================================================================
  END OF TRANSCRIPT
================================================================
```